

Copia de Diseño 5 - ACTUAL

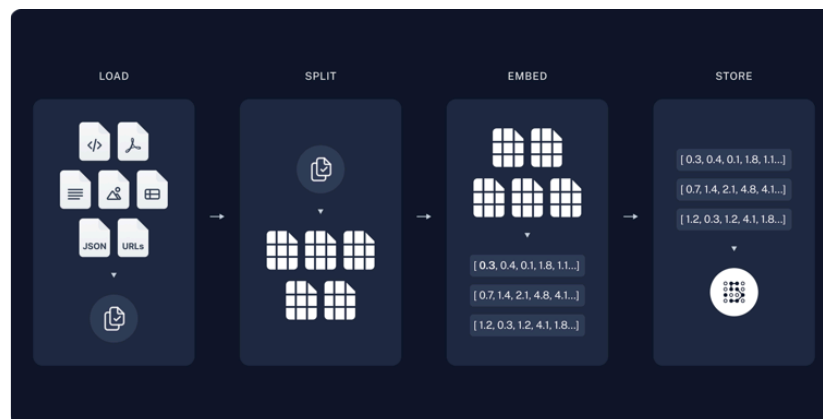
Diseño del flujo de Retrieval-Augmented Generation (RAG)

El componente central de la solución desarrollada se basa en la arquitectura **Retrieval-Augmented Generation**, un enfoque que combina técnicas de recuperación de información (*Information Retrieval*) con modelos de lenguaje de gran tamaño (*Large Language Models*, LLM). El objetivo principal de este paradigma es complementar las capacidades generativas del modelo con información proveniente de una base documental previamente indexada, permitiendo que las respuestas sean construidas a partir de conocimiento verificable y no únicamente del conocimiento adquirido durante el entrenamiento del modelo.

En aplicaciones institucionales como la propuesta para la FAU, la utilización de un enfoque RAG resulta especialmente relevante debido a la naturaleza sensible y específica de la información utilizada. Los modelos de lenguaje, por sí solos, pueden generar respuestas plausibles pero incorrectas, fenómeno conocido como *hallucination* o alucinación. Al incorporar un mecanismo de recuperación documental, el modelo dispone de evidencia proveniente exclusivamente de la documentación autorizada almacenada en la plataforma, reduciendo significativamente la probabilidad de generar información incorrecta y permitiendo mantener la trazabilidad de las respuestas.

Desde el punto de vista arquitectónico, el sistema RAG implementado se organiza en dos procesos claramente diferenciados: el proceso de **indexación documental**, ejecutado cuando nuevos documentos son incorporados a la plataforma, y el proceso de **recuperación y generación**, ejecutado cada vez que un usuario realiza una consulta. Esta separación permite desacoplar las tareas de procesamiento intensivo del flujo interactivo de consulta, optimizando el rendimiento general del sistema.

Etapa de indexación documental



La indexación constituye la primera fase del sistema RAG y tiene como finalidad transformar documentos no estructurados en una representación semántica que pueda ser consultada eficientemente durante las búsquedas posteriores. Este proceso se ejecuta automáticamente cada vez que un usuario incorpora un nuevo documento al repositorio documental.

La arquitectura implementada divide esta etapa en una secuencia de procesos especializados, donde cada uno cumple una responsabilidad específica dentro del pipeline de procesamiento.

Carga y normalización de documentos

La primera etapa consiste en la incorporación del documento al sistema y la extracción de su contenido textual. La plataforma admite múltiples formatos de documentos, entre ellos PDF y DOCX, permitiendo que la información institucional pueda ser incorporada independientemente de su formato original.

Durante esta etapa el contenido es normalizado con el objetivo de obtener una representación homogénea que pueda ser procesada posteriormente. La normalización comprende tareas como la eliminación de elementos irrelevantes, la corrección de problemas derivados del formato del documento, la unificación de la codificación de caracteres y la limpieza del texto extraído. Como resultado, todo el contenido documental es transformado en texto plano, independizando el resto del pipeline del formato de origen.

Este proceso resulta fundamental para garantizar que documentos provenientes de diferentes fuentes puedan ser tratados de manera uniforme, reduciendo la variabilidad introducida por las distintas aplicaciones con las que fueron originalmente generados.

Fragmentación del contenido

Una vez normalizado el documento, el contenido es dividido en fragmentos de menor tamaño (*chunks*). Esta etapa responde a una limitación inherente tanto de los modelos de lenguaje como de los modelos de embeddings, los cuales operan sobre ventanas de contexto de longitud finita.

La fragmentación persigue múltiples objetivos. En primer lugar, mejora la precisión de la recuperación de información al permitir localizar únicamente las secciones relevantes del documento, evitando recuperar documentos completos cuando solo una pequeña parte contiene la respuesta buscada. En segundo lugar, incrementa la granularidad de las búsquedas semánticas, permitiendo que cada fragmento represente una unidad coherente de información. Finalmente, reduce el costo computacional tanto del proceso de indexación como de las consultas posteriores.

Desde una perspectiva arquitectónica, esta decisión permite que el sistema escale de forma eficiente incluso cuando el repositorio documental contiene documentos de gran extensión.

Generación de embeddings

Cada uno de los fragmentos generados es posteriormente transformado en una representación vectorial mediante un modelo de *embeddings*. A diferencia de una representación textual tradicional, un embedding codifica el significado semántico del contenido dentro de un espacio vectorial multidimensional, permitiendo medir la similitud conceptual entre distintos textos.

Gracias a esta representación, el sistema puede recuperar información por similitud de significado en lugar de depender únicamente de coincidencias exactas de palabras clave. Esto resulta especialmente importante cuando diferentes documentos expresan un mismo concepto utilizando terminología distinta, sinónimos o construcciones lingüísticas diferentes.

La utilización de embeddings constituye uno de los elementos fundamentales del enfoque RAG, ya que establece el vínculo entre la información documental almacenada y las consultas formuladas posteriormente por los usuarios.

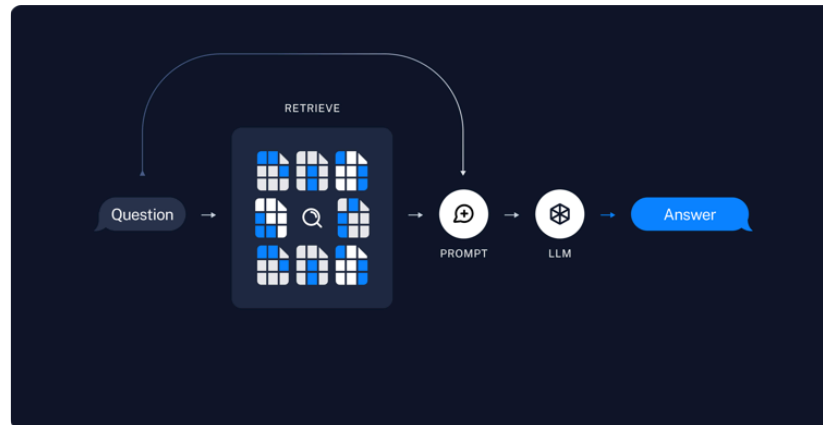
Almacenamiento e indexación vectorial

Una vez generados los embeddings, estos son almacenados junto con sus respectivos fragmentos documentales dentro de la base de datos vectorial del sistema. Para esta implementación se optó por utilizar la extensión **pgvector** sobre PostgreSQL, lo que permite integrar el almacenamiento relacional tradicional con capacidades de búsqueda vectorial dentro de una única plataforma tecnológica.

Esta decisión arquitectónica simplifica significativamente la infraestructura del sistema, evitando incorporar motores vectoriales independientes y reduciendo la complejidad operativa del despliegue. Al mismo tiempo, permite mantener conjuntamente la información documental, los metadatos asociados y las representaciones vectoriales, favoreciendo la consistencia de los datos y simplificando las tareas de administración y respaldo.

Como resultado de esta etapa, cada documento queda completamente indexado y preparado para participar en el proceso de recuperación semántica que será utilizado durante la generación de respuestas.

Etapa de recuperación y generación



Una vez que los documentos han sido indexados, el sistema se encuentra preparado para responder consultas formuladas por los usuarios. Cada vez que se recibe una nueva consulta, se ejecuta el proceso de **recuperación y generación** (*Retrieval-Augmented Generation*), cuya finalidad es localizar la información más relevante dentro del repositorio documental y utilizarla como contexto para la generación de una respuesta mediante un modelo de lenguaje.

A diferencia de un chatbot tradicional, donde el modelo responde únicamente a partir del conocimiento adquirido durante su entrenamiento, en un sistema RAG la respuesta se construye utilizando información recuperada dinámicamente desde la base documental. Este enfoque permite mantener las respuestas alineadas con la documentación institucional, incorporar conocimiento actualizado sin necesidad de reentrenar el modelo y reducir significativamente la generación de información incorrecta o no fundamentada.

La etapa de recuperación y generación se compone de dos procesos principales: la recuperación semántica de información y la generación condicionada de la respuesta.

Recuperación semántica

El primer paso consiste en transformar la consulta formulada por el usuario en una representación vectorial utilizando el mismo modelo de embeddings empleado durante el proceso de indexación. De esta manera, tanto los documentos como las consultas se encuentran representados dentro del mismo espacio semántico, haciendo posible comparar su significado mediante medidas de similitud.

Una vez obtenido el embedding de la consulta, el sistema realiza una búsqueda sobre la base vectorial para identificar los fragmentos documentales cuya representación resulta más cercana desde el punto de vista semántico. A diferencia de una búsqueda tradicional basada en palabras clave, este mecanismo permite recuperar información conceptualmente relacionada incluso cuando la consulta utiliza una terminología diferente a la empleada en los documentos originales.

Como resultado de este proceso se obtiene un conjunto ordenado de fragmentos relevantes que constituyen la evidencia documental sobre la cual será construida la respuesta. Estos fragmentos representan el contexto que posteriormente será suministrado al modelo de lenguaje, garantizando que la generación se apoye exclusivamente en información disponible dentro del repositorio institucional.

Desde el punto de vista arquitectónico, esta etapa constituye el principal mecanismo mediante el cual el sistema conecta el conocimiento almacenado con las capacidades generativas del modelo, permitiendo responder consultas específicas sobre grandes volúmenes de documentación sin necesidad de procesar nuevamente todos los documentos.

Generación condicionada mediante el modelo de lenguaje

Una vez recuperado el contexto documental, el sistema construye el mensaje que será enviado al modelo de lenguaje. Este mensaje, comúnmente denominado *prompt*, combina diferentes elementos que orientan el proceso de generación y establecen el comportamiento esperado del modelo.

En primer lugar, se incorpora la consulta realizada por el usuario. A continuación, se agregan los fragmentos documentales recuperados durante la etapa de búsqueda semántica, los cuales proporcionan la información necesaria para responder la consulta. Finalmente, se incluyen instrucciones específicas que definen el comportamiento del modelo, estableciendo aspectos como el formato de la respuesta, el estilo de redacción, las restricciones de contenido y la obligación de fundamentar la respuesta utilizando únicamente la información proporcionada como contexto.

Esta estrategia convierte al modelo de lenguaje en un mecanismo de síntesis y razonamiento sobre la evidencia recuperada, en lugar de un generador de respuestas basado exclusivamente en su conocimiento previo. En consecuencia, las respuestas obtenidas presentan una mayor coherencia con la documentación institucional, reducen la probabilidad de alucinaciones y ofrecen un mayor nivel de confiabilidad para aplicaciones donde la precisión de la información resulta crítica.

La separación entre la recuperación de información y la generación de respuestas constituye uno de los principales beneficios del enfoque RAG. Mientras que la base documental puede actualizarse de forma continua mediante el proceso de indexación, el modelo de lenguaje permanece inalterado, permitiendo incorporar nuevo conocimiento al sistema sin necesidad de reentrenar el modelo. Esta característica aporta flexibilidad, facilita el mantenimiento de la solución y garantiza que las respuestas reflejen siempre el contenido más reciente disponible en el repositorio documental.

Relevancia del enfoque RAG en el contexto del proyecto

La adopción de una arquitectura basada en **RAG** constituye una de las decisiones de diseño más importantes de la solución propuesta, ya que responde tanto a los requisitos funcionales como a las restricciones técnicas y operativas del proyecto.

En primer lugar, la plataforma está orientada al tratamiento de documentación institucional propia de la Fuerza Aérea Uruguaya. Esta información no forma parte del conocimiento con el que fueron entrenados los modelos de lenguaje y, además, se encuentra en constante evolución. Como consecuencia, un modelo de lenguaje utilizado de manera aislada no podría responder consultas específicas sobre esta documentación con un nivel adecuado de precisión ni garantizar que las respuestas reflejen el contenido vigente de los documentos institucionales. El enfoque RAG resuelve esta limitación al recuperar dinámicamente la información relevante desde la base documental e incorporarla al contexto del modelo durante el proceso de generación, permitiendo que las respuestas se fundamenten en fuentes autorizadas sin necesidad de reentrenar el modelo.

Por otra parte, la solución fue diseñada para ejecutarse íntegramente sobre infraestructura local, sin dependencia de servicios externos y utilizando modelos de lenguaje de tamaño moderado que puedan operar con los recursos computacionales disponibles. En este escenario, la ventana de contexto del modelo constituye un recurso limitado. Aunque los modelos modernos admiten contextos cada vez mayores, incorporar documentos completos en cada consulta resulta inviable cuando el repositorio documental crece, tanto por las restricciones de memoria como por el incremento en el tiempo de inferencia y el consumo de recursos.

En este contexto, el enfoque RAG permite optimizar significativamente el proceso de inferencia. En lugar de enviar documentos completos al modelo de lenguaje, el sistema recupera únicamente aquellos fragmentos cuya similitud semántica indica que son relevantes para responder la consulta planteada. Esta estrategia reduce considerablemente la cantidad de tokens procesados por el modelo, disminuye los tiempos de respuesta y posibilita utilizar modelos más pequeños ejecutándose localmente, sin sacrificar la calidad de las respuestas.

Adicionalmente, la separación entre el conocimiento documental y el modelo de lenguaje aporta importantes ventajas desde el punto de vista de la mantenibilidad del sistema. La incorporación, modificación o eliminación de documentos únicamente requiere actualizar el índice vectorial, manteniendo inalterado el modelo de inteligencia artificial. De esta forma, la base de conocimiento puede evolucionar continuamente sin necesidad de realizar costosos procesos de reentrenamiento, permitiendo que la plataforma refleje de manera inmediata los cambios en la documentación institucional.

En consecuencia, la utilización de una arquitectura RAG no solo mejora la precisión y confiabilidad de las respuestas generadas, sino que también constituye un elemento fundamental para hacer viable el despliegue de la solución en un entorno local, con recursos computacionales limitados y sobre un repositorio documental en permanente actualización. Estas características hacen que este enfoque resulte especialmente adecuado para el contexto operativo y los requisitos planteados por la Fuerza Aérea Uruguaya.

Procesamiento de documentos mediante Docling

Uno de los desafíos principales de un sistema RAG consiste en transformar documentos de diferentes formatos en una representación textual homogénea que pueda ser procesada por el resto del pipeline. En un entorno institucional, es habitual que la información se encuentre distribuida en documentos PDF, archivos de Microsoft Word, presentaciones, hojas de cálculo y otros formatos, cada uno con estructuras y características particulares.

Con el objetivo de unificar este proceso, la arquitectura incorpora **Docling** como componente encargado de la extracción y estructuración del contenido documental. Esta herramienta proporciona una interfaz común para el procesamiento de múltiples formatos de archivo, permitiendo obtener una representación consistente del contenido independientemente del documento de origen.

Además de la extracción del texto, Docling preserva gran parte de la estructura lógica del documento, identificando elementos como títulos, encabezados, secciones, listas, tablas e imágenes. Esta información resulta especialmente valiosa para las etapas posteriores del pipeline, ya que permite mantener la organización original del documento y facilita la generación de fragmentos coherentes desde el punto de vista semántico.

Otro aspecto relevante es que Docling permite tratar de forma uniforme documentos con distintos niveles de complejidad, incluyendo aquellos que contienen tablas, diagramas o estructuras jerárquicas que habitualmente presentan dificultades para los métodos tradicionales de extracción de texto. Gracias a ello, el sistema obtiene una representación documental de mayor calidad, mejorando tanto el proceso de indexación como la recuperación posterior de información.

La utilización de Docling como etapa inicial del procesamiento documental proporciona una base sólida para el resto del flujo RAG, ya que garantiza que todos los documentos ingresen al sistema bajo una representación estructurada y consistente, independientemente de su formato original. Esto incrementa la calidad de la información indexada y contribuye a mejorar la precisión de las respuestas generadas por el sistema.

Metadatos documentales

Además del contenido textual y de sus representaciones vectoriales, durante el proceso de indexación se extraen metadatos estructurales asociados a cada fragmento del documento. Para ello se utiliza un mecanismo de fragmentación basado en **Docling**, que preserva la estructura lógica del documento durante el proceso de extracción.

Estos metadatos incluyen información como la página de origen, títulos, secciones, subsecciones y otros elementos estructurales que permiten mantener la relación entre cada fragmento y su ubicación dentro del documento original. De esta manera, el sistema no solo recupera el contenido más relevante desde el punto de vista semántico, sino también el contexto documental al que pertenece.

La conservación de esta información aporta múltiples beneficios al proceso RAG. En primer lugar, permite mejorar la trazabilidad de las respuestas generadas, ya que el sistema puede identificar con precisión el origen de la información utilizada. En segundo lugar, facilita la presentación de referencias al usuario, indicando el documento, la sección o la página desde la cual fue obtenida la evidencia documental. Finalmente, estos metadatos enriquecen el proceso de recuperación al incorporar información estructural que puede utilizarse para priorizar o contextualizar los fragmentos recuperados.

La utilización de metadatos documentales constituye, por tanto, un complemento al proceso de recuperación semántica, permitiendo que las respuestas generadas no solo sean precisas, sino también verificables y fácilmente rastreables hasta su fuente original.

Enriquecimiento de embeddings mediante contexto documental

Como se describió anteriormente, durante el proceso de indexación cada fragmento generado a partir de la etapa de *chunking* es transformado en una representación vectorial (*embedding*). Esta representación captura el significado semántico del contenido del fragmento y constituye la base del proceso de recuperación semántica utilizado por el sistema RAG.

Sin embargo, uno de los desafíos habituales de este enfoque es que, al fragmentar documentos de gran tamaño, cada *chunk* pierde parte del contexto global del documento al que pertenece. En consecuencia, algunos fragmentos pueden contener información cuya interpretación depende de secciones anteriores o posteriores del documento, dificultando que el modelo de *embeddings* represente adecuadamente su significado de forma aislada.

Con el objetivo de mitigar esta limitación, la arquitectura incorpora una etapa adicional de **enriquecimiento semántico de embeddings** durante el postprocesamiento documental. Una vez finalizada la indexación inicial, el documento completo es enviado al servicio de inteligencia artificial, donde un modelo de lenguaje genera un resumen que sintetiza el contenido general y los principales conceptos desarrollados en el documento.

Posteriormente, este resumen es asociado a cada uno de los fragmentos generados durante la etapa de *chunking*. A partir de esta nueva representación enriquecida —compuesta por el contenido original del fragmento más el resumen global del documento— se calcula un segundo embedding que incorpora tanto la información local del fragmento como el contexto general del documento del cual proviene.

Como resultado de este proceso, cada fragmento dispone de dos representaciones vectoriales complementarias. La primera corresponde al contenido original del fragmento y refleja con precisión la información específica contenida en esa sección del documento. La segunda incorpora el contexto proporcionado por el resumen, permitiendo representar de manera más completa el significado del fragmento dentro del documento al que pertenece.

Durante el proceso de recuperación semántica, ambas representaciones son utilizadas para identificar los fragmentos más relevantes para una consulta. Esta estrategia permite combinar la precisión del contenido específico de cada fragmento con una representación enriquecida del contexto global, incrementando la probabilidad de recuperar información pertinente cuando la consulta hace referencia a conceptos generales del documento o cuando el significado del fragmento depende del contexto en el que fue escrito.

Desde el punto de vista arquitectónico, este enfoque mejora la calidad del proceso de recuperación sin modificar el funcionamiento general del pipeline RAG. La etapa de enriquecimiento se ejecuta únicamente durante la indexación, por lo que el costo adicional de generar los resúmenes y calcular los embeddings enriquecidos se incurre una sola vez por documento. En cambio, todas las consultas posteriores se benefician de una representación semántica más completa, mejorando la calidad del contexto proporcionado al modelo de lenguaje y, en consecuencia, la precisión de las respuestas generadas por la plataforma.

Recuperación léxica mediante BM25

Si bien la búsqueda semántica basada en *embeddings* constituye el principal mecanismo de recuperación de información de la plataforma, esta presenta ciertas limitaciones en escenarios donde la consulta contiene términos muy específicos, nombres propios, códigos, siglas o expresiones cuya coincidencia textual resulta determinante para obtener resultados precisos. Para complementar este comportamiento, la arquitectura incorpora un mecanismo de recuperación léxica basado en **BM25 (Best Matching 25)**.

BM25 es uno de los algoritmos más utilizados en los sistemas modernos de recuperación de información y representa una evolución del modelo probabilístico clásico de búsqueda textual. Su funcionamiento se basa en calcular un puntaje de relevancia entre una consulta y cada documento considerando factores como la frecuencia con la que aparecen los términos buscados, la importancia relativa de dichos términos dentro del conjunto documental y la longitud de cada documento. De esta manera, el algoritmo favorece aquellos fragmentos donde los términos de la consulta aparecen con mayor relevancia, evitando al mismo tiempo que los documentos más extensos reciban una ventaja injustificada.

A diferencia de la búsqueda semántica, BM25 no intenta comprender el significado del texto, sino que evalúa la coincidencia entre las palabras presentes en la consulta y las contenidas en los documentos. Esta característica resulta especialmente útil cuando los usuarios realizan consultas que incluyen denominaciones oficiales, identificadores, nomenclaturas técnicas, códigos militares, números de documentos o terminología específica cuya coincidencia exacta es fundamental para recuperar la información correcta.

Dentro del flujo de recuperación implementado en la plataforma, BM25 actúa como un mecanismo complementario a la búsqueda vectorial. Ambos enfoques generan de forma independiente un conjunto inicial de fragmentos candidatos: mientras la búsqueda semántica recupera contenido conceptualmente relacionado con la consulta, BM25 prioriza aquellos fragmentos que contienen coincidencias textuales relevantes. Posteriormente, los resultados obtenidos por ambos mecanismos son unificados y enviados a la etapa de *reranking*, donde se determina cuáles representan la evidencia documental más adecuada para responder la consulta.

La combinación de recuperación semántica y recuperación léxica permite aprovechar las fortalezas de ambos enfoques. La búsqueda vectorial facilita encontrar información relacionada desde el punto de vista del significado, incluso cuando la terminología utilizada difiere entre la consulta y los documentos. Por su parte, BM25 garantiza que las coincidencias textuales importantes no sean descartadas, aspecto especialmente relevante en documentación técnica e institucional donde pequeñas diferencias en nombres, códigos o identificadores pueden modificar completamente el significado de la información recuperada.

En consecuencia, la incorporación de BM25 mejora la robustez del proceso de recuperación al proporcionar una mayor diversidad de candidatos antes de la etapa de selección final. Esta estrategia híbrida incrementa la probabilidad de recuperar los fragmentos verdaderamente relevantes y constituye una práctica ampliamente adoptada en sistemas RAG modernos, donde la combinación de recuperación léxica y recuperación semántica permite obtener mejores resultados que la utilización de cualquiera de ambos enfoques de forma aislada.

Uso de *rerankers* en el proceso de recuperación

Si bien la búsqueda semántica permite identificar los fragmentos documentales más similares a una consulta, la similitud vectorial por sí sola no siempre garantiza que los resultados recuperados sean los más relevantes desde el punto de vista del significado o de la intención del usuario. Por este motivo, la arquitectura incorpora una etapa adicional de **reranking**, cuyo objetivo es refinar el conjunto de resultados antes de que estos sean utilizados por el modelo de lenguaje.

Un *reranker* es un modelo especializado que recibe una consulta junto con un conjunto de documentos o fragmentos previamente recuperados y estima el grado de relevancia de cada uno de ellos respecto de dicha consulta. A diferencia de los modelos de *embeddings*, que representan tanto la consulta como los documentos mediante vectores

independientes para posteriormente calcular una medida de similitud, los *rerankers* analizan simultáneamente ambos textos. Esto les permite comprender mejor la relación existente entre la pregunta y cada fragmento, obteniendo una estimación de relevancia considerablemente más precisa.

Dentro del flujo implementado en la solución, el *reranker* actúa como una segunda etapa de recuperación (*second-stage retrieval*). Inicialmente, el sistema obtiene un conjunto amplio de candidatos mediante mecanismos de recuperación eficientes, entre ellos la búsqueda vectorial y un método de recuperación léxica basado en BM25. Posteriormente, todos los fragmentos recuperados son enviados al modelo de *reranking*, el cual asigna una puntuación de relevancia a cada uno y los reordena de acuerdo con dicha evaluación.

Este enfoque permite trabajar con un número elevado de fragmentos durante la fase inicial de recuperación sin aumentar innecesariamente el contexto enviado al modelo de lenguaje. En lugar de incorporar todos los documentos candidatos, únicamente se seleccionan aquellos que obtienen la mayor puntuación de relevancia, reduciendo la cantidad de información procesada por el LLM y aprovechando de forma más eficiente la ventana de contexto disponible.

La incorporación de un *reranker* aporta múltiples beneficios al sistema. En primer lugar, incrementa la precisión de la recuperación documental al reducir la presencia de fragmentos poco relevantes dentro del contexto suministrado al modelo. En segundo lugar, disminuye la probabilidad de que información irrelevante interfiera en el proceso de generación de respuestas, mejorando la calidad y coherencia de las respuestas obtenidas. Finalmente, optimiza el uso de los recursos computacionales al limitar el contexto enviado al modelo de lenguaje únicamente a la información con mayor valor para la consulta realizada.

En aplicaciones RAG sobre repositorios documentales extensos, la utilización de un *reranker* se ha convertido en una práctica ampliamente adoptada, ya que combina la velocidad de los métodos tradicionales de recuperación con la mayor precisión que ofrecen los modelos de reordenamiento. De esta forma, la arquitectura implementada logra un equilibrio entre eficiencia y calidad de recuperación, proporcionando al modelo de lenguaje un contexto más preciso y representativo antes de iniciar el proceso de generación de la respuesta.

Observabilidad del pipeline de inteligencia artificial mediante Phoenix

Además de los mecanismos tradicionales de monitoreo utilizados en la arquitectura, el sistema incorpora una capa de observabilidad específica para los componentes de inteligencia artificial mediante **Arize Phoenix**. A diferencia de las herramientas de observabilidad convencionales, orientadas al monitoreo de infraestructura y aplicaciones, Phoenix está diseñado para analizar el comportamiento interno de aplicaciones basadas en modelos de lenguaje, agentes y arquitecturas RAG.

Durante la ejecución del pipeline, Phoenix registra las distintas etapas que intervienen en el procesamiento de una consulta, proporcionando trazabilidad completa sobre el recorrido realizado por la información. Entre otros aspectos, permite visualizar los *prompts* enviados al modelo de lenguaje, los fragmentos recuperados durante la búsqueda semántica, las decisiones tomadas por los agentes, las respuestas generadas y los tiempos de ejecución de cada etapa.

Esta capacidad resulta especialmente útil durante las fases de desarrollo y evaluación del sistema, ya que facilita la identificación de problemas relacionados con la recuperación de contexto, la construcción de *prompts* o el comportamiento del modelo de lenguaje. Asimismo, permite analizar la calidad de las respuestas generadas y detectar posibles oportunidades de optimización dentro del pipeline RAG.

La incorporación de Phoenix complementa la observabilidad tradicional de la plataforma al proporcionar una visión detallada del funcionamiento de los componentes de inteligencia artificial. De esta manera, es posible no solo monitorear el estado operativo de la aplicación, sino también comprender cómo el sistema recupera información, procesa el contexto y genera las respuestas, facilitando el diagnóstico, la validación y la mejora continua del flujo de inteligencia artificial.

Grafo de conocimiento mediante Neo4j

Como complemento al mecanismo tradicional de recuperación basado en embeddings, la arquitectura incorpora un **grafo de conocimiento** implementado sobre Neo4j, cuyo objetivo es representar explícitamente las entidades presentes en la documentación institucional y las relaciones existentes entre ellas. Mientras que la búsqueda vectorial permite localizar información similar desde el punto de vista semántico, el grafo de conocimiento modela la estructura del dominio, posibilitando consultas centradas en las conexiones entre personas, organizaciones, ubicaciones, unidades militares, documentos, procedimientos y cualquier otra entidad relevante identificada durante el procesamiento documental.

Los grafos de conocimiento constituyen una de las principales técnicas utilizadas para representar información estructurada dentro de sistemas de inteligencia artificial. A diferencia de una base de datos relacional, donde la información se organiza en tablas, un grafo representa el conocimiento mediante **nodos** (entidades) y **relaciones** (vínculos entre entidades). Este modelo resulta especialmente adecuado para dominios donde las relaciones poseen tanto valor como los propios datos, permitiendo responder consultas complejas que requieren recorrer múltiples conexiones entre distintos elementos del conocimiento.

En el contexto del proyecto, la utilización de un grafo de conocimiento permite complementar las capacidades del sistema RAG tradicional. Existen consultas cuya respuesta no depende únicamente del contenido textual de los documentos, sino también de las relaciones existentes entre los distintos conceptos presentes en ellos. Por ejemplo, identificar qué unidades participaron en una determinada operación, qué documentos hacen referencia a un mismo procedimiento, o qué personas estuvieron involucradas en un determinado evento requiere comprender las conexiones entre múltiples entidades distribuidas en distintos documentos. Estas consultas resultan difíciles de resolver únicamente mediante búsqueda semántica, mientras que un grafo permite representarlas y recorrerlas de forma natural.

Pipeline de construcción del grafo

La construcción del grafo forma parte del pipeline de procesamiento documental y se ejecuta una vez que el contenido del documento ha sido extraído y normalizado. El objetivo de este proceso es transformar información no estructurada en una representación explícita del conocimiento contenido en los documentos.

La primera etapa consiste en el análisis del contenido mediante un modelo de lenguaje, encargado de identificar las entidades relevantes presentes en el texto. Dependiendo del dominio documental, estas entidades pueden corresponder a personas, organizaciones, ubicaciones geográficas, unidades militares, cargos, documentos, equipos, fechas, operaciones o cualquier otro concepto significativo para el contexto institucional.

Una vez identificadas las entidades, el modelo determina las relaciones existentes entre ellas. Estas relaciones representan el significado estructural del documento y describen cómo interactúan los distintos elementos identificados. Algunos ejemplos incluyen relaciones de pertenencia, participación, dependencia jerárquica, ubicación, autoría, utilización o referencia entre entidades.

Posteriormente, las entidades y relaciones extraídas son transformadas en una estructura compatible con el modelo de grafos e incorporadas a Neo4j. Durante este proceso se aplican mecanismos de normalización destinados a evitar la creación de entidades duplicadas y mantener la consistencia del conocimiento almacenado. De esta forma, una misma entidad mencionada en múltiples documentos queda representada mediante un único nodo, enriquecido progresivamente con nuevas relaciones a medida que se procesa documentación adicional.

Como resultado de este pipeline, el conocimiento originalmente distribuido en numerosos documentos independientes pasa a conformar una red interconectada de entidades y relaciones, facilitando su exploración y análisis.

Utilización del grafo durante las consultas

Una vez construido el grafo de conocimiento, este puede ser utilizado como una fuente adicional de información durante el proceso de recuperación. Cuando una consulta requiere comprender relaciones entre entidades, el sistema puede interpretar la intención del usuario y generar automáticamente consultas en **Cypher**, el lenguaje de consulta de Neo4j, para obtener la información estructurada necesaria.

Los resultados obtenidos desde el grafo pueden emplearse de manera independiente o combinarse con la recuperación semántica tradicional basada en embeddings. Mientras que la búsqueda vectorial aporta evidencia textual proveniente de los documentos originales, el grafo proporciona información estructurada acerca de las conexiones existentes entre las distintas entidades. Esta combinación permite enriquecer el contexto suministrado al modelo de lenguaje y ampliar el tipo de consultas que la plataforma es capaz de responder.

Beneficios de la integración

La incorporación de un grafo de conocimiento aporta múltiples ventajas dentro de la arquitectura propuesta. En primer lugar, mejora la representación del conocimiento institucional al transformar información dispersa en una estructura organizada y fácilmente explorable. En segundo lugar, permite responder consultas cuya resolución depende del análisis de relaciones explícitas entre entidades y no únicamente de la similitud textual de los documentos.

Asimismo, el grafo facilita la reutilización del conocimiento extraído durante el procesamiento documental, evitando que el modelo de lenguaje deba inferir repetidamente relaciones que ya fueron identificadas previamente. Finalmente, la integración entre recuperación semántica y conocimiento estructurado constituye una estrategia ampliamente utilizada en sistemas modernos de inteligencia artificial, ya que combina las fortalezas del enfoque RAG tradicional con las capacidades de razonamiento proporcionadas por los grafos de conocimiento.

De esta manera, la arquitectura incorpora dos mecanismos complementarios para la recuperación de información: uno basado en similitud semántica mediante representaciones vectoriales y otro basado en relaciones explícitas representadas en un grafo. La combinación de ambos enfoques proporciona una representación más rica del conocimiento institucional y amplía significativamente las capacidades analíticas de la plataforma.

Procesamiento de documentos extensos mediante Map-Reduce

Si bien el enfoque RAG resulta adecuado para responder consultas específicas sobre un conjunto reducido de fragmentos documentales, existen determinadas operaciones que requieren comprender el contenido de un documento en su totalidad. Entre ellas se encuentran la generación de resúmenes, la elaboración de reportes, la clasificación documental y otras tareas de análisis global, donde limitar el procesamiento a unos pocos fragmentos recuperados no resulta suficiente.

Una de las principales limitaciones de los modelos de lenguaje locales es el tamaño de su ventana de contexto. Aunque los modelos actuales admiten contextos cada vez mayores, documentos extensos pueden superar fácilmente dicha capacidad, imposibilitando su procesamiento en una única solicitud. Además, enviar documentos completos al modelo incrementa significativamente el consumo de memoria, el tiempo de inferencia y el costo computacional, aspectos especialmente relevantes en un entorno de ejecución local como el planteado en este proyecto.

Para resolver esta limitación, la arquitectura implementa una estrategia basada en el paradigma **Map-Reduce**, ampliamente utilizado en el procesamiento distribuido de grandes volúmenes de datos y adaptado en este caso al procesamiento mediante modelos de lenguaje.

La primera etapa, denominada **Map**, consiste en dividir el documento en múltiples fragmentos utilizando el mismo proceso de segmentación empleado durante la indexación. Cada uno de estos fragmentos es procesado de manera independiente por el modelo de lenguaje, generando un resultado parcial acorde a la tarea solicitada. Dependiendo del objetivo, estos

resultados pueden corresponder a resúmenes parciales, extracción de información relevante, identificación de conceptos clave o cualquier otro procesamiento definido por la aplicación.

Al procesar cada fragmento de forma independiente, el sistema evita las restricciones impuestas por la ventana de contexto del modelo y permite trabajar con documentos de cualquier tamaño sin comprometer la calidad del procesamiento.

Una vez obtenidos todos los resultados parciales, se ejecuta la etapa **Reduce**, cuyo propósito es combinar la información generada durante la fase anterior para construir un único resultado coherente. En esta etapa, el modelo recibe los resultados producidos por cada procesamiento parcial y genera una síntesis global que integra la información relevante proveniente de todo el documento.

Este enfoque permite que el modelo analice progresivamente documentos de gran extensión sin necesidad de procesarlos completos en una única inferencia. Asimismo, reduce el consumo de recursos computacionales y facilita el procesamiento de documentación extensa utilizando modelos de lenguaje de tamaño moderado ejecutándose sobre infraestructura local.

Dentro de la solución desarrollada, esta estrategia se emplea en aquellas funcionalidades que requieren una comprensión global del contenido documental. Entre ellas se encuentran la generación de resúmenes, la elaboración de reportes, la creación de listas de verificación (*checklists*), la clasificación de documentos y otras tareas donde resulta necesario analizar el documento completo en lugar de responder una consulta puntual.

Desde el punto de vista arquitectónico, la utilización del paradigma Map-Reduce permite desacoplar el tamaño del documento de las limitaciones del modelo de lenguaje. De esta manera, la plataforma puede procesar documentos considerablemente más extensos que la ventana de contexto disponible, manteniendo tiempos de respuesta razonables y haciendo viable el uso de modelos de inteligencia artificial locales con recursos computacionales limitados.

Protección de información sensible mediante NeMo Guardrails

Uno de los aspectos fundamentales de la plataforma es garantizar que las consultas realizadas por los usuarios y las respuestas generadas por el sistema respeten las políticas de seguridad establecidas por la organización. Para ello, la arquitectura incorpora **NVIDIA NeMo Guardrails** como mecanismo de control y validación de las interacciones con los modelos de lenguaje.

Los *guardrails* constituyen una capa intermedia entre el usuario y el modelo de lenguaje cuya finalidad es supervisar el flujo de información antes y después del proceso de inferencia. Su objetivo es asegurar que tanto las solicitudes recibidas como las respuestas generadas cumplan con las reglas de funcionamiento definidas para la aplicación, reduciendo riesgos asociados al uso de modelos generativos.

Dentro de la solución desarrollada, NeMo Guardrails interviene en distintos puntos del pipeline de conversación. Antes de que una consulta sea procesada por el modelo de lenguaje, el sistema analiza el mensaje del usuario para detectar situaciones que puedan comprometer la seguridad o el correcto funcionamiento de la plataforma. Entre ellas se incluyen intentos de manipulación del modelo (*prompt injection*), solicitudes destinadas a ignorar las instrucciones internas del sistema, consultas fuera del alcance funcional de la aplicación y otros comportamientos que puedan afectar la confiabilidad de las respuestas.

Una vez generada la respuesta, NeMo Guardrails realiza una segunda etapa de validación sobre el contenido producido por el modelo. En esta fase se verifica que la respuesta respete las restricciones establecidas para la plataforma, evitando la generación de información no autorizada, contenido inapropiado o respuestas que contradigan las políticas definidas por la organización.

La incorporación de esta capa de protección resulta especialmente relevante en aplicaciones RAG destinadas al manejo de documentación institucional. Si bien el mecanismo de recuperación documental limita el contexto utilizado por el modelo, continúa existiendo la posibilidad de que usuarios malintencionados intenten modificar el comportamiento del sistema mediante técnicas de *prompt injection* o consultas diseñadas para eludir las restricciones del asistente. La utilización de guardrails proporciona una defensa adicional frente a este tipo de ataques, aumentando la robustez y confiabilidad de la solución.

Asimismo, el uso de NeMo Guardrails permite centralizar las políticas de comportamiento del asistente en un único componente de la arquitectura. Esto facilita la incorporación de nuevas reglas de seguridad, la modificación de las existentes y la adaptación del comportamiento del sistema a futuros requerimientos sin necesidad de alterar la lógica principal de los agentes o del modelo de lenguaje.

En conjunto, la incorporación de NeMo Guardrails fortalece la seguridad del sistema de inteligencia artificial al añadir mecanismos de validación antes y después de la inferencia, contribuyendo a que las respuestas generadas se mantengan alineadas con los objetivos funcionales de la plataforma y con las políticas de seguridad definidas para el entorno institucional.

Orquestación mediante agentes

La arquitectura del sistema incorpora un enfoque basado en **agentes**, cuyo objetivo es estructurar el proceso de razonamiento del modelo de lenguaje en una secuencia de tareas especializadas. En lugar de resolver todas las consultas mediante un único flujo de ejecución, el sistema organiza el procesamiento como un conjunto de etapas interconectadas, donde cada una posee una responsabilidad específica dentro del pipeline de inteligencia artificial.

Este enfoque permite descomponer problemas complejos en tareas más pequeñas y controladas, facilitando la incorporación de nuevas funcionalidades y mejorando la mantenibilidad de la solución. Asimismo, favorece la reutilización de componentes, ya que un mismo nodo puede formar parte de distintos flujos de trabajo dependiendo del tipo de consulta que deba resolverse.

La implementación se basa en un modelo de ejecución representado mediante un **grafo de nodos**, donde cada nodo corresponde a una etapa del procesamiento y las conexiones entre ellos definen el flujo de ejecución. Cada nodo recibe un estado compartido, realiza una operación específica y devuelve un estado actualizado que será utilizado por los nodos posteriores. De esta manera, la información va enriqueciéndose progresivamente a medida que avanza por el flujo.

Dentro de la plataforma, los nodos implementan distintas responsabilidades según el tipo de operación a realizar. Algunos se encargan de analizar la consulta del usuario, otros recuperan información documental, ejecutan búsquedas, construyen el contexto que será enviado al modelo de lenguaje, generan la respuesta o realizan tareas de postprocesamiento antes de devolver el resultado al usuario. Esta separación permite que cada componente permanezca desacoplado del resto, simplificando tanto su desarrollo como su evolución futura.

Otro aspecto importante de esta arquitectura es la posibilidad de incorporar **decisiones condicionales** dentro del flujo de ejecución. En función del contenido de la consulta o del estado alcanzado durante el procesamiento, el sistema puede seleccionar distintos caminos de ejecución. Por ejemplo, una consulta puede derivarse hacia un flujo RAG tradicional, hacia un agente con herramientas externas, hacia un proceso de generación de reportes o hacia otro flujo especializado, reutilizando gran parte de los componentes existentes.

La representación del procesamiento mediante grafos también facilita la incorporación de ciclos controlados, validaciones intermedias y etapas de verificación antes de producir la respuesta final. Esto permite implementar estrategias de razonamiento más sofisticadas que las obtenidas mediante una única llamada al modelo de lenguaje, incrementando la robustez del sistema y proporcionando un mayor control sobre el comportamiento de la inteligencia artificial.

Desde el punto de vista de la arquitectura, la utilización de agentes proporciona un elevado grado de modularidad. La incorporación de una nueva funcionalidad generalmente implica agregar nuevos nodos o definir un nuevo flujo de ejecución, sin necesidad de modificar los agentes existentes. Esto favorece la escalabilidad funcional del sistema y simplifica el mantenimiento a medida que la plataforma evoluciona.

En el contexto del proyecto, este enfoque resulta especialmente adecuado debido a la diversidad de tareas soportadas por la plataforma. Funcionalidades como la consulta sobre documentos mediante RAG, la generación de resúmenes, la elaboración de reportes, la creación de listas de verificación, la clasificación documental o la interacción con herramientas externas comparten una misma infraestructura de ejecución basada en nodos especializados, permitiendo construir flujos adaptados a cada necesidad sin duplicar lógica de procesamiento.

La adopción de una arquitectura basada en agentes representa, por tanto, una evolución respecto a los pipelines lineales tradicionales. Al modelar el procesamiento como un grafo de nodos interconectados, la plataforma obtiene un mayor nivel de flexibilidad, reutilización y capacidad de adaptación, características especialmente relevantes en aplicaciones modernas basadas en modelos de lenguaje y sistemas de Retrieval-Augmented Generation.

Diseño de las APIs

Todos los servicios del sistema exponen interfaces REST para permitir la comunicación entre el frontend, el API Gateway y los distintos microservicios. Las APIs siguen una estructura homogénea, utilizando versionado mediante prefijos en las rutas y una organización consistente de los recursos, lo que facilita su mantenimiento, escalabilidad y evolución.

Asimismo, cada servicio genera automáticamente su documentación OpenAPI, permitiendo consultar de forma interactiva los contratos de las APIs durante el desarrollo y las tareas de integración.

Desde el punto de vista de su implementación, los microservicios se estructuran siguiendo una **arquitectura en capas (Layered Architecture)**, lo que permite una clara separación de responsabilidades y un mayor desacoplamiento entre los componentes del sistema.

En este enfoque, se distinguen las siguientes capas:

La **capa de controladores (Controller / API layer)** constituye el punto de entrada de las solicitudes HTTP. Su responsabilidad es recibir las peticiones del cliente, validar los parámetros de entrada, gestionar aspectos relacionados con la autenticación y autorización cuando corresponde, y delegar el procesamiento a la capa de servicios. De esta forma, los controladores se mantienen livianos y enfocados exclusivamente en la interacción con el protocolo HTTP.

La **capa de servicios (Service layer)** concentra la lógica de negocio del sistema. En ella se implementan las reglas funcionales de la aplicación, la orquestación de procesos y la coordinación entre distintos componentes o microservicios. Esta separación evita que la lógica de negocio se distribuya entre controladores o repositorios, favoreciendo su reutilización, testeabilidad y mantenimiento.

Por debajo se encuentra la **capa de repositorios (Repository layer)**, encargada del acceso a la persistencia. Esta capa encapsula las operaciones sobre las bases de datos u otras fuentes de datos, proporcionando una interfaz uniforme para la lectura, escritura y actualización de la información. De este modo, la lógica de negocio permanece independiente de la tecnología de almacenamiento utilizada.

Finalmente, la **capa de modelos (Domain / Model layer)** representa las entidades del dominio de la aplicación. Estas entidades reflejan los conceptos centrales del sistema, tales como usuarios, documentos, colecciones, conversaciones, permisos o notificaciones, y constituyen la base estructural sobre la cual opera el resto de la arquitectura.

Esta organización en capas se aplica de manera consistente en todos los microservicios del sistema, independientemente de la tecnología utilizada para su implementación. Tanto los servicios desarrollados con Django como aquellos

implementados en FastAPI mantienen esta misma separación de responsabilidades, lo que permite preservar una arquitectura uniforme, facilitando el desarrollo de nuevas funcionalidades, las tareas de prueba y el mantenimiento evolutivo de la plataforma.

Diseño de las bases de datos

El sistema utiliza una arquitectura de datos distribuida basada en dos bases de datos principales: **auth-db**, destinada exclusivamente al servicio de autenticación y gestión de usuarios, y **aura-db**, que concentra la persistencia del resto de los dominios funcionales del sistema. Esta separación permite aislar las responsabilidades relacionadas con la seguridad y el control de acceso del resto de la lógica operativa de la plataforma.

La base de datos **auth-db** almacena la información relacionada con usuarios, roles, permisos, tokens de autenticación y registros de auditoría. Este esquema sigue un modelo relacional clásico de control de acceso basado en roles (RBAC), donde los usuarios pueden pertenecer a múltiples roles y estos, a su vez, agrupan permisos específicos. Además, se incluye soporte para gestión de sesiones mediante refresh tokens y mecanismos de trazabilidad mediante logs de auditoría.

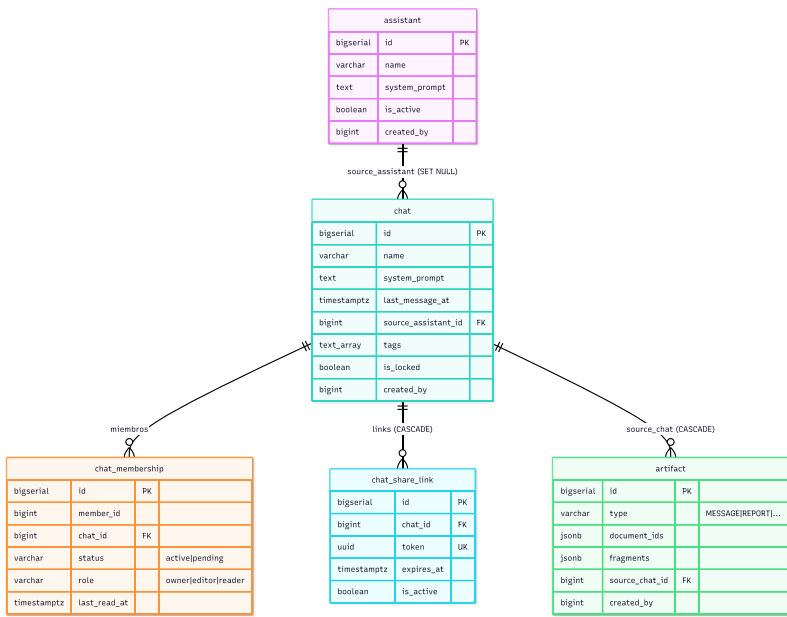
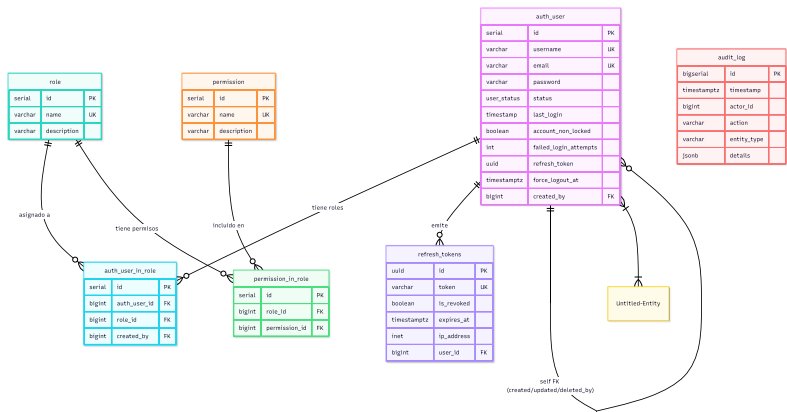
Por otro lado, la base de datos **aura-db** integra varios dominios funcionales del sistema dentro de una única instancia física, aunque organizados lógicamente por módulos correspondientes a microservicios. Entre estos dominios se incluyen la gestión de chats y artefactos, el procesamiento de documentos, la organización de colecciones con controles de clasificación, y el sistema de notificaciones.

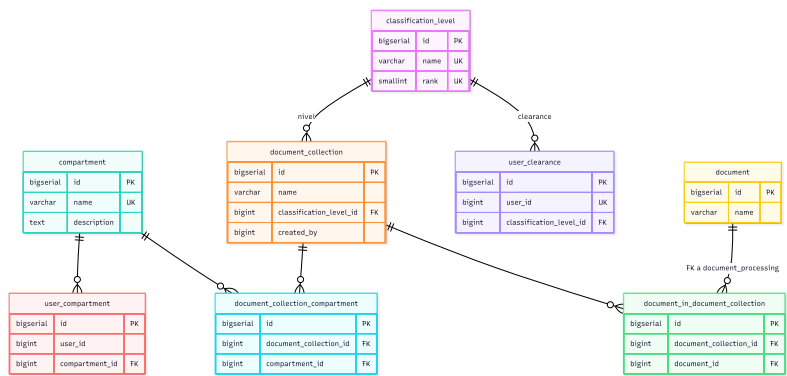
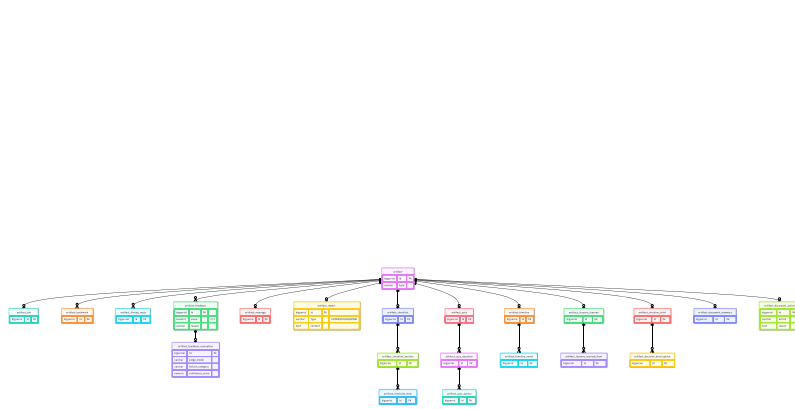
El dominio de **document processing** introduce soporte para almacenamiento y segmentación de documentos, incluyendo fragmentación de contenido y generación de embeddings vectoriales, lo que permite capacidades de búsqueda semántica. Asimismo, el dominio de **document collections** implementa un modelo de control de acceso basado en niveles de clasificación y compartimentación de información, orientado a escenarios con requerimientos de seguridad más estrictos.

El sistema de **notificaciones** gestiona eventos internos y comunicaciones hacia los usuarios, manteniendo independencia lógica respecto del resto de los módulos. En este caso, las referencias a usuarios se realizan de forma lógica hacia la base de autenticación, sin imponer integridad referencial a nivel de base de datos, reforzando así la independencia entre servicios.

En términos de diseño, se observa que **auth-db actúa como frontera de identidad del sistema**, mientras que el resto de los dominios en aura-db comparten una base de datos común pero mantienen separación conceptual mediante esquemas lógicos. Existen algunas relaciones entre dominios dentro de aura-db, como la vinculación entre documentos, chats y colecciones, lo que implica una dependencia en el orden de creación de las tablas durante la inicialización del sistema.

En conjunto, este diseño busca equilibrar la consistencia de datos con la flexibilidad de una arquitectura orientada a microservicios, permitiendo mantener un núcleo de identidad centralizado y, al mismo tiempo, una estructura modular para los distintos dominios funcionales.





notification			
bigserial	id	PK	
bigint	receiver_id		
varchar	event_type		
varchar	message		
jsonb	data		
varchar	severity		info success warning critical
varchar	status		unread read
timestampz	read_at		

email_dispatch			
bigserial	id	PK	
bigint	receiver_id		
varchar	event_type		
varchar	status		pending sent failed skipped
int	attempt		
jsonb	payload		
timestampz	sent_at		

notification_preference			
bigint	user_id	PK	
boolean	inapp_enabled		
boolean	email_enabled		
timestampz	mute_until		
bigint	user_id	PK	
boolean	inapp_enabled		
boolean	email_enabled		
timestampz	mute_until		

Diseño de seguridad

El sistema implementa un diseño de seguridad estructurado en múltiples capas, que combinan mecanismos de autenticación, autorización y control de acceso contextual. Esta estrategia permite garantizar tanto la integridad del proceso de autenticación como la correcta segmentación de permisos y la protección de la información en función del contexto operativo.

En primer lugar, el proceso de autenticación se basa en el uso de tokens de acceso de tipo JSON Web Token (JWT), firmados mediante el algoritmo HS256. Estos tokens contienen únicamente la información mínima necesaria para identificar al usuario autenticado y validar su estado, incluyendo su identificador único, su condición de superadministrador y su fecha de expiración. Con el objetivo de mantener la seguridad y reducir el riesgo asociado a la exposición prolongada de credenciales, los tokens de acceso tienen una vida útil corta.

La verificación de permisos no se resuelve de forma estática dentro del token, sino que se delega a un servicio central de validación de autenticación. Este servicio provee información actualizada sobre la identidad del usuario, sus roles y los permisos asociados, lo que permite mantener consistencia ante cambios dinámicos en las autorizaciones sin depender de tokens previamente emitidos.

Además, el sistema incorpora un mecanismo de renovación de sesión basado en tokens de refresco opacos. Estos tokens se almacenan de forma persistente en la base de datos y cuentan con un período de expiración definido. Su gestión sigue una política de rotación, en la cual cada uso invalida el token anterior y genera uno nuevo, reduciendo significativamente el riesgo de reutilización indebida. Asimismo, el cierre de sesión implica la revocación inmediata del token activo, asegurando el control completo sobre la finalización de sesiones.

Como parte de las medidas de protección frente a accesos no autorizados, el sistema incorpora controles de bloqueo temporal de cuentas ante múltiples intentos fallidos de autenticación consecutivos. Este mecanismo introduce una restricción temporal al acceso cuando se detectan patrones de intento de fuerza bruta. Adicionalmente, antes de permitir cualquier operación de autenticación, se valida el estado general de la cuenta, verificando que esta se encuentre activa, habilitada y no bloqueada. Todas las acciones relevantes dentro del proceso de autenticación son registradas en un sistema de auditoría inmutable, lo que permite trazabilidad completa de eventos de seguridad.

La autorización se maneja mediante un modelo de roles (RBAC): cada usuario tiene roles asignados, y esos roles agrupan los permisos que determinan qué puede hacer dentro de la plataforma. Esto evita tener que codificar privilegios caso por caso.

caso y hace más simple incorporar cambios sin tocar la lógica del sistema.

Dentro de los chats, la granularidad va más allá del RBAC general. Cada participante tiene un nivel de acceso propio a la conversación: puede ser propietario, editor o lector, y eso define concretamente qué operaciones puede realizar. También existe la posibilidad de bloquear un chat para impedir que se agreguen nuevos mensajes. Para incorporar usuarios externos, el sistema usa enlaces de invitación con fecha de vencimiento configurable que pueden revocarse en cualquier momento.

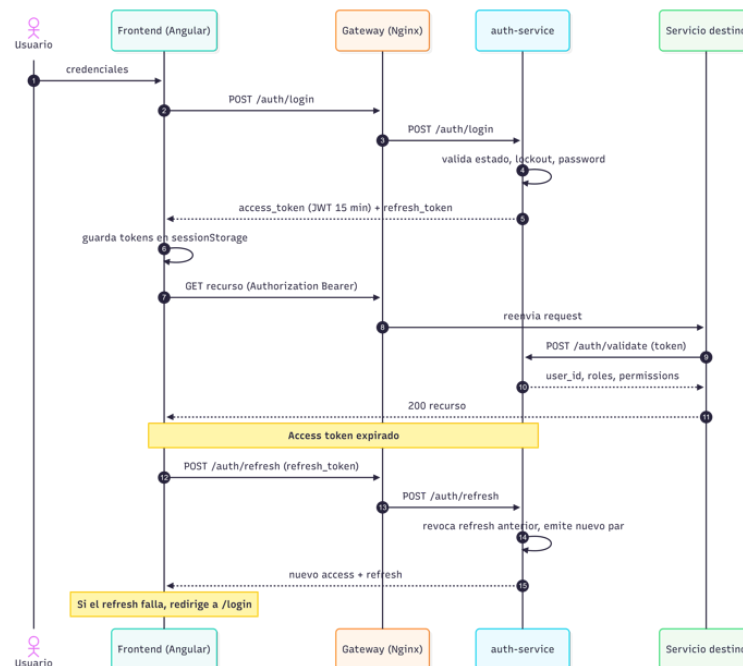
Por último, el sistema incorpora un modelo avanzado de control de acceso a la información basado en principios de seguridad obligatoria, diseñado para entornos institucionales. Este modelo define una estructura jerárquica de clasificación de la información, donde cada recurso se encuentra asociado a un nivel de sensibilidad. Para complementar, cada usuario posee un nivel de habilitación que determina su capacidad máxima de acceso, así como un conjunto de compartimentos que representan dominios específicos de información autorizada.

El acceso a los recursos se concede únicamente cuando el nivel de clasificación del contenido no supera el nivel de habilitación del usuario y, simultáneamente, cuando el usuario posee los compartimentos requeridos por dicho recurso. Este enfoque permite implementar una política estricta de control de acceso basada en el principio de “necesidad de conocer”, evitando la exposición de información sensible a usuarios no autorizados.

Este modelo de control se extiende también al proceso de recuperación de información en el sistema de generación aumentada por recuperación (RAG), garantizando que únicamente los documentos autorizados sean utilizados como contexto para la generación de respuestas. De este modo, se asegura que la información sensible no sea expuesta ni directa ni indirectamente a usuarios sin los permisos correspondientes.

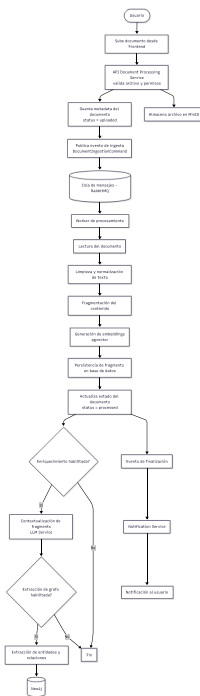
Algunos flujos de la aplicación

Autenticación y validación de solicitudes



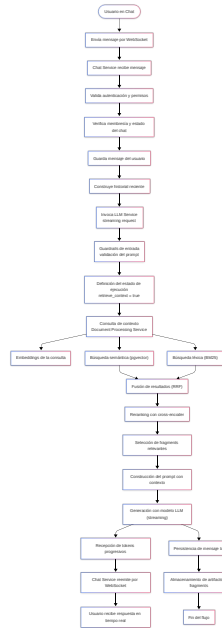
El proceso de autenticación se inicia cuando el usuario introduce sus credenciales en el sistema. El servicio de autenticación verifica en primer lugar el estado de la cuenta, asegurando que se encuentre activa, habilitada y no bloqueada. Una vez validada esta condición, se emite un token de acceso de corta duración junto con un token de refresco con capacidad de revocación.

En el lado del backend, cada microservicio no confía únicamente en la validez estructural del token, sino que realiza una verificación adicional contra un servicio central de autenticación. Este servicio expone un punto de validación que retorna la identidad del usuario junto con sus roles y permisos vigentes, garantizando así que los cambios en la autorización se reflejen de manera inmediata en todas las solicitudes.



Adicionalmente, el sistema contempla un proceso de enriquecimiento de los fragmentos, que puede incluir la extracción de entidades relevantes y la construcción de estructuras auxiliares para un eventual grafo de conocimiento. Una vez finalizado el procesamiento, el sistema emite un evento de notificación que es consumido por el servicio correspondiente, permitiendo informar al usuario sobre la disponibilidad del documento procesado en tiempo real.

Consulta RAG en un chat



El proceso de consulta en el sistema de chat se realiza a través de una conexión WebSocket, lo que permite comunicación en tiempo real entre el usuario y el sistema. Cuando el usuario envía un mensaje, este es persistido por el servicio de chat y difundido inmediatamente a los demás participantes de la conversación.

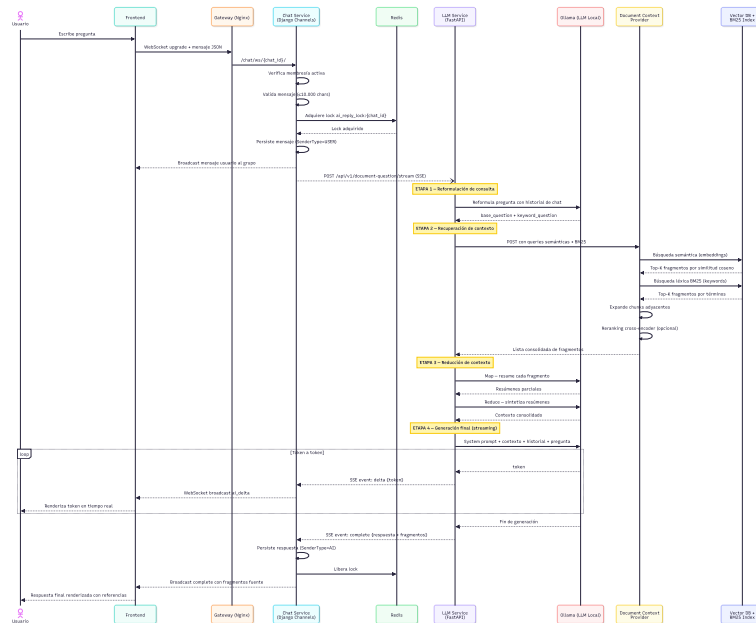
Para garantizar la consistencia en la generación de respuestas, el sistema implementa un mecanismo de control de concurrencia que evita la generación simultánea de múltiples respuestas para un mismo chat. Este control se gestiona mediante un candado distribuido, asegurando que solo una solicitud de generación pueda ser procesada a la vez.

Posteriormente, el servicio de chat delega la solicitud al servicio de modelos de lenguaje, el cual selecciona el modo de ejecución correspondiente. Este servicio realiza la recuperación de información relevante desde el sistema de procesamiento documental, aplicando búsquedas semánticas sobre los fragmentos almacenados y filtrando los resultados según los permisos de acceso del usuario.

Con el contexto recuperado, se construye el prompt que será enviado al modelo de lenguaje, el cual genera la respuesta final. Esta respuesta es almacenada como un mensaje del sistema dentro del chat, incluyendo las referencias a los fragmentos utilizados como soporte. Luego, la respuesta es difundida a todos los participantes activos de la conversación.

Finalmente, el sistema libera el control de concurrencia, permitiendo nuevas solicitudes de generación. El usuario tiene la posibilidad de evaluar la respuesta recibida o solicitar su regeneración. En caso de que el chat se encuentre en estado bloqueado, el sistema rechaza nuevas entradas de mensajes, indicando que la conversación se encuentra restringida.

Flujo Completo de una Consulta



El flujo completo de una consulta comienza cuando el usuario introduce una pregunta en la interfaz web. Esta consulta es enviada desde el frontend, desarrollado en React, hacia el backend mediante una conexión WebSocket, donde el mensaje es serializado en formato JSON e incluye metadatos que determinan el comportamiento del sistema, como el modo de operación, flags de recuperación de contexto y opciones de procesamiento de documentos.

Una vez enviada la solicitud, el primer componente en recibirla es el gateway basado en Nginx, que funciona como punto único de entrada al sistema. Este componente se encarga de enrutar las peticiones hacia los distintos microservicios según el prefijo de la URL, gestionar el upgrade de conexiones WebSocket, aplicar políticas de rate limiting diferenciadas y definir timeouts extendidos para operaciones de inferencia, especialmente aquellas relacionadas con el modelo de lenguaje.

Posteriormente, la solicitud es recibida por el Chat Service, implementado sobre Django Channels, el cual gestiona la comunicación en tiempo real y la coherencia del estado del chat. En esta etapa se realizan validaciones de seguridad y consistencia, incluyendo la verificación de la membresía activa del usuario, la validación del tamaño del mensaje y el estado del chat. Adicionalmente, se implementa un mecanismo de control de concurrencia mediante un lock distribuido en Redis asociado al identificador del chat, lo que garantiza que no se generen respuestas simultáneas de IA dentro del mismo contexto conversacional. Una vez validadas estas condiciones, el mensaje del usuario se persiste en la base de datos y se emite a todos los clientes conectados al canal WebSocket correspondiente. En paralelo, se lanza una tarea asíncrona independiente encargada de invocar al servicio de generación de lenguaje, desacoplando así la interacción del usuario del proceso de inferencia.

El LLM Service constituye el núcleo del sistema y es responsable de la generación de respuestas mediante un pipeline de recuperación aumentada por contexto. Este pipeline inicia con una etapa de reformulación de la consulta, en la cual el modelo genera dos versiones de la pregunta original: una versión semánticamente enriquecida, orientada a la comprensión del significado global, y otra basada en palabras clave, optimizada para la búsqueda léxica. Esta separación permite mejorar la calidad de la recuperación en etapas posteriores.

A continuación, se realiza la recuperación de contexto a través del Document Context Provider, donde se aplica una estrategia híbrida de búsqueda. Por un lado, se utiliza búsqueda semántica basada en embeddings y similitud coseno, y por otro, búsqueda léxica mediante el algoritmo BM25. Ambos enfoques se complementan, ya que el primero captura similitud conceptual mientras que el segundo garantiza precisión en términos exactos, como nombres propios o términos técnicos. Adicionalmente, se incorporan fragmentos adyacentes a los resultados relevantes con el fin de preservar coherencia contextual, y opcionalmente se aplica un reranking mediante modelos cross-encoder para refinar la relevancia de los documentos recuperados.

Cuando el volumen de información recuperada excede la ventana de contexto del modelo, el sistema aplica una estrategia de reducción basada en MapReduce. En la fase Map, cada fragmento es resumido de manera independiente por el modelo, extrayendo únicamente la información relevante para la consulta. En la fase Reduce, estos resúmenes parciales son integrados en un único contexto coherente que será utilizado en la generación final. Este mecanismo permite escalar el sistema a grandes volúmenes de información sin comprometer la capacidad del modelo para procesarlos.

Con el contexto consolidado, el sistema construye el prompt final incorporando el system prompt, el contexto recuperado, el historial reciente de la conversación y la pregunta del usuario. A partir de este input, el modelo genera la respuesta utilizando streaming token a token, lo que permite reducir significativamente la latencia percibida por el usuario y ofrecer una experiencia interactiva en tiempo real.

Durante este proceso, el Chat Service actúa como intermediario entre el LLM Service y los clientes conectados. Este componente recibe eventos estructurados mediante Server-Sent Events, los cuales pueden corresponder a metadatos iniciales, progreso del pipeline, tokens generados, finalización de la respuesta o errores. Cada token generado es transmitido inmediatamente al frontend a través del WebSocket, mientras que el evento final se utiliza para persistir la respuesta completa en la base de datos y liberar los recursos asociados al lock distribuido.

Finalmente, el frontend consume estos eventos y actualiza la interfaz de manera reactiva. Los tokens recibidos se van concatenando progresivamente, generando el efecto de escritura en tiempo real característico de los sistemas conversacionales modernos. Una vez recibida la respuesta final, el contenido es normalizado y renderizado en su versión definitiva, incluyendo formato enriquecido y referencias a documentos cuando corresponde.

Bibliografía

LangChain. (2024). *Build a Retrieval Augmented Generation (RAG) App: Part 1*. Recuperado de <https://python.langchain.com/docs/tutorials/rag/>

TigerData. (2025). *PostgreSQL as a Vector Database: A Pgvector Tutorial*. Recuperado de <https://www.tigerdata.com/blog/postgresql-as-a-vector-database-using-pgvector>

IBM Research. (2023). *What is retrieval-augmented generation (RAG)?*. Recuperado de <https://research.ibm.com/blog/retrieval-augmented-generation-RAG>

IBM. (2025). *GenAI RAG: Generative AI with Retrieval-Augmented Generation*. Recuperado de https://www.ibm.com/mx-es/architectures/hybrid/genai-rag?mhsrc=ibmsearch_a&mhg=rag

Amazon Web Services (AWS). (2025). *What is Retrieval-Augmented Generation?*. Recuperado de <https://aws.amazon.com/what-is/retrieval-augmented-generation/>

Microsoft. (2025). *Retrieval-Augmented Generation in Azure AI Search*. Recuperado de <https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview>

pgvector. (s.f.). *pgvector: Open-source vector similarity search for Postgres*. GitHub. Recuperado el 12 de septiembre de 2025, de <https://github.com/pgvector/pgvector>

